
GI41 高度ゲームアルゴリズム

第26回 講義テーマ

圧縮プログラミング (1)

講義内容

1. ハフマン符号による圧縮
2. 次回授業連絡(講義予定・持参物等)

1. ハフマン符号による圧縮

1. ハフマン符号による圧縮

ハフマン符号とは、1952年にDavid A. Huffmanによって開発された符号である。コンパクト符号やエントロピー符号の一つ。シャノン符号化が最適ではない場合が存在する不完全な符号であったのに対し、ハフマン符号は常に最適な符号を構成できる。

1. ハフマン符号による圧縮

算術符号やその他の高効率の符号化法と異なり、特許の問題が無い。

そのため、JPEGやLHAなどの圧縮フォーマットで使用されている。

1. ハフマン符号による圧縮

(符号化の原理)

(1) まずデータ群に出現する記号(個々のデータ)の個数を求める。

それがツリー構造の葉に相当すると見なし、ボトムアップで木を構成する。

<元データの例>

「aabcdeafagagfedadhaeedefahhh」

数えると a=8, b=1, c=1, d=4, e=5, f=3, g=2, h=4 となる。

1. ハフマン符号による圧縮

(2) 葉を含むすべての節点のうち、親を持たないものを集める。その中から「最小の値をもつもの」と「2番目に小さい値をもつもの」を取り出す。

＜例＞最小の値と2番目に小さい値 … b
と c が該当 (共に1)

1. ハフマン符号による圧縮

(3) それらを子供にもつ新しい節点を作る。このとき、新しい節点の値は「両方の子供の値の和」とする。

1. ハフマン符号による圧縮

(4) 以上を繰り返して根節点(ルート)まで到達して木を完成させる。

1. ハフマン符号による圧縮

(5) 根から順に左右に0と1の値を割り振っていく
(左右のどちらに0と1を与えるかは任意)。すると、それぞれの葉(記号)に対して、一意にビット列が与えられる。

1. ハフマン符号による圧縮

a = 00

e = 10

b = 11111

f = 110

c = 11110

g = 1110

d = 011

h = 010

1. ハフマン符号による圧縮

(6) この記号とビット列の関係を使い、元データの記号をビット列に変換していくことで符号化を行う。

1. ハフマン符号による圧縮

(7) ファイルに記録するときは、ビット列と元データとの変換テーブルを同時に記録する。

1. ハフマン符号による圧縮

＜例＞のデータをビット列に変換すると・・・

a	a	b		c		d		e	a	f		a	g		a	g		f		e	d		a	d		h		a	e	e	d		e	f		a	h		h		h				
0	0	0	0	1	1	1	1	1	1	1	1	1	0	0	1	1	1	0	0	0	1	1	0	0	0	1	1	1	0	1	1	0	1	0	0	1	1	0	0	0	1	0	0	1	0

つまり、このように「77ビット」で表現できることがわかる。ファイルにはバイト単位で記録するので「10バイト」あればOKであることが分かる。元データが28文字＝28バイトなので、半分以下に圧縮できたことになる。

1. ハフマン符号による圧縮

(複合化の手順)

- (1) ビット列と元データとの変換テーブルを用意する。
 - (2) テーブルのビット列からツリーを作成する。
 - (3) 圧縮データの先頭からビットを順次読み出し、ツリーをたどる。
 - (4) ツリーの葉つまり末端に到達したらそこにあるデータ(文字コード)を出力する。
 - (5) これを繰り返し、元データを復元する。
-